Stage 3 Transformer Project for EML Load Forecasting

1. Motivation

The EML Lab has developed an AI-powered load forecasting system that corresponds to **Stage 2** of our three-stage framework. The current system uses customized machine learning to estimate an interpretable econometric model without oversimplification. It is built around load, temperature, and calendar variables, and it supports counterfactual decomposition, self-diagnosis, and self-revision.

The next step is to build the first **Stage 3 extension**. Stage 3 is designed to incorporate richer information streams that do not fit naturally into the conventional Stage 2 model structure. These may include textual signals, real-time operational notices, energy-market news, weather alerts, policy announcements, and other unconventional data sources.

The goal of this project is to begin building a **transformer-based residual learner** that augments the existing EML load forecaster.

2. Core Principle: Augment, Do Not Replace

We do **not** want to replace the existing forecasting system. The Stage 2 model is already valuable because it is interpretable, stable, and grounded in economic and energy-domain structure.

Instead, we will treat the existing forecast as the baseline and train a transformer to explain and predict the remaining forecast error.

The basic structure is:

$$y_t = \hat{y}_t^{Stage2} + r_t,$$

where:

$$r_t = y_t - \hat{y}_t^{Stage2}.$$

The transformer will try to predict r_t, the residual or forecast error, using information that the Stage 2 model may not fully capture.

Thus the final forecast becomes:

$$\hat{y}_t^{Final} = \hat{y}_t^{Stage2} + \hat{r}_t^{Transformer}.$$

This keeps the Stage 2 model as the interpretable forecasting core while allowing Stage 3 to add information from text and real-time signals.

3. Why a Residual Learner?

The residual learner has several advantages.

First, it protects the interpretability of the current model. The Stage 2 forecast remains the baseline, and the transformer only provides an adjustment.

Second, it gives us a clean way to measure the value added by Stage 3. If the transformer reduces residual variance or improves MAPE, we can show that the new information stream adds predictive value beyond the existing model.

Third, it allows us to evaluate whether the current Stage 2 model has already absorbed most of the relevant structure. We can compare models with and without the original covariates to see whether the transformer is learning genuinely new information or merely rediscovering what Stage 2 already captured.

4. Two Model Versions to Compare

We should begin with three benchmark models.

Model 0: Existing Stage 2 forecast

This is the current EML load forecaster.

$$\hat{y}_t^{Stage2}.$$

This is the baseline.

Model 1: Text-only residual learner

This model predicts the residual using only textual information.

$$r_t = f(\text{text embeddings}_t) + u_t.$$

This asks:

Does textual information alone help explain the forecast errors of the existing model?

Model 2: Text plus selected covariates

This model predicts the residual using textual information plus a small number of conventional covariates, such as temperature or calendar indicators.

$$r_t = f(\text{text embeddings}_t, X_t) + u_t.$$

This asks:

Does textual information help conditionally, especially when combined with temperature or day-type information?

Model 3: Full feature-fusion model, later stage

Eventually, we may also consider a model that directly uses all covariates and text to forecast load. But this should not be the first step because it risks replacing the interpretable Stage 2 core with a less interpretable black-box model.

5. Data Structure for the First Prototype

For the first experiment, we only need a small prototype dataset. The transformer will be trained to predict the **residual**. The actual load and Stage 2 forecast should still be retained in the dataset, not as primary model inputs at first, but to reconstruct the final forecast and evaluate improvement relative to the Stage 2 baseline.

The key relationship is:

$$\text{residual} = \text{actual load} - \text{Stage 2 forecast}.$$

The transformer target is:

$$\text{target} = \text{residual}.$$

The final forecast is then reconstructed as:

$$\text{final forecast} = \text{Stage 2 forecast} + \text{predicted residual adjustment}.$$

A prototype dataset should include the following variables:

timestamp
actual_load
stage2_forecast
residual
temperature
day_type
text_source
raw_text

The most important variables for the first prototype are:

timestamp
stage2_forecast
actual_load
residual
raw_text

The variable residual is the object the transformer learns to predict. The variables actual_load and stage2_forecast are retained so that we can compute the final adjusted forecast and compare accuracy before and after the transformer correction.

A simple CSV-style example is:

Example observation 1

timestamp: 2025-07-04 14:00
actual_load: ...
stage2_forecast: ...
residual: ...
temperature: 94
day_type: holiday
text_source: MISO alert
raw_text: Severe weather alert issued...

Example observation 2

timestamp: 2025-07-04 15:00
actual_load: ...
stage2_forecast: ...
residual: ...
temperature: 95
day_type: holiday
text_source: news
raw_text: Midwest heat wave raises demand...

This format is much easier to work with than a wide table in Word. Later, we can add more covariates or additional text fields if needed.

6. Textual Data Sources

The most useful textual data should be timestamped and related to grid stress, weather, or energy-market conditions.

Possible sources include:

MISO operational notifications

These may include:

- severe weather alerts,
- maximum generation warnings,
- transmission constraints,
- reliability alerts,
- operational notices.

These are likely to be highly relevant because they are directly connected to grid operations.

Weather alerts

Examples include:

- National Weather Service alerts,
- heat wave warnings,
- cold weather advisories,
- storm warnings.

These may be especially useful when forecast errors occur during unusual weather events.

Energy news

Examples include:

- Utility Dive,
- EIA updates,
- regional energy news,
- local news about outages, policy changes, or demand surges.

Broader public text datasets

Eventually, we may consider larger datasets such as GDELT or other news/event databases, but we should begin with a small, controlled set of highly relevant textual data.

7. Turning Text into Numerical Inputs

The transformer cannot directly read raw text in a regression-like format. The text must first be converted into numerical representations.

There are two useful approaches.

A. Sentiment or event scores

A model can convert text into a simple score, such as:

- grid stress score,
- severe weather score,
- uncertainty score,
- sentiment score.

This is more interpretable.

Example:

raw_text: Severe weather alert issued
stress_score: 0.85

B. Text embeddings

A sentence-transformer or language model can convert the text into a high-dimensional vector that captures the meaning of the text.

Example:

$$\text{text embedding}_t = (0.12, -0.44, 0.03, \dots).$$

This is less directly interpretable but may capture richer information.

A good first strategy is to use both:

- a simple interpretable text score,
- and a richer embedding vector.

Then we can evaluate whether the richer embedding adds value beyond the simple score.

8. Role of the Transformer

The transformer should be understood as the Stage 3 information-integration layer.

It receives sequences of information over time. For example:

- residuals from recent hours or days,
- recent text embeddings,
- recent weather alerts,
- recent operational notices,
- selected conventional covariates.

The transformer then learns which parts of the recent information history matter most for predicting the next residual.

This is where the attention mechanism matters. On a normal day, the transformer may give little weight to text. On a grid-stress day, it may place more weight on operational alerts or weather-related language.

9. Interpretability Strategy

The transformer itself is not fully interpretable internally. But our design preserves external interpretability in three ways.

First, the baseline forecast is still produced by the Stage 2 model.

Second, the transformer adjustment is separated from the baseline forecast:

$$\hat{y}_t^{Final} = \hat{y}_t^{Stage2} + \hat{r}_t^{Transformer}.$$

This lets us explain how much of the final forecast comes from the existing model and how much comes from the textual-information adjustment.

Third, we can inspect attention weights, text scores, and error reductions around specific events. For example, we can ask whether the transformer adjustment becomes large during heat waves, severe weather alerts, or MISO operational warnings.

10. Initial Implementation Plan

The first implementation should be small and simple.

Step 1: Create a prototype dataset

Use 50–100 observations first.

Include:

- timestamp,
- Stage 2 forecast,
- actual load,
- residual,
- one or two conventional covariates,
- one text column.

Step 2: Convert text to embeddings

Use a sentence-transformer to convert raw_text into numerical vectors.

Possible libraries:

- sentence-transformers,
- transformers,
- torch.

Step 3: Train a simple residual model

Start with a simple model before a full transformer:

- linear regression on text scores,
- random forest on embeddings,
- small neural network.

This gives a baseline for whether the textual information has signal.

Step 4: Train a small transformer

Only after the simple benchmarks should we train a small transformer that uses sequences.

Step 5: Compare models

Compare:

- Stage 2 forecast only,
- Stage 2 plus text-only residual adjustment,
- Stage 2 plus text and covariate residual adjustment.

Evaluation metrics:

- residual MSE,

- residual MAE,
- final forecast MAPE,
- improvement on special days,
- improvement during high-stress events.

11. Cursor AI Workflow

We will use Cursor AI as the coding environment because it allows us to build and modify a multi-file research project interactively.

Suggested project folder:

```
eml_transformer/
  data/
  models/
  training/
  evaluation/
  text_data/
  utils/
  notebooks/
```

Suggested first libraries:

```
pip install pandas numpy matplotlib scikit-learn torch sentence-transformers transformers
```

Possible first Cursor prompt:

I am building a research prototype for electricity load forecasting. I have a CSV with timestamp, actual_load, stage2_forecast, residual, temperature, day_type, and raw_text. Please write a Python script that loads the CSV, uses sentence-transformers to convert raw_text into embeddings, and trains a simple model to predict residuals. Then compare final forecast accuracy before and after adding the predicted residual adjustment.

A later Cursor prompt:

Please extend this project by creating a small PyTorch transformer model that uses sequences of text embeddings and selected covariates to predict the next residual. Keep the code modular, with separate files for data loading, model definition, training, and evaluation.

12. Student Tasks

Ralph, Aiden, and Jack

Ralph, Aiden, and Jack will form the main summer project team. Their work will cover both data and modeling.

Initial tasks include:

- organize the eml_transformer project folder;
- connect Stage 2 forecast outputs and residuals to the new residual-learning pipeline;
- collect timestamped textual data relevant to MISO, weather, grid stress, and energy-market conditions;
- process raw text into structured text variables and embeddings;
- compare residual-learning models using text only and text plus selected covariates;
- develop evaluation metrics for forecast improvement over the Stage 2 baseline;
- document the codebase and maintain reproducible workflows.

Useful topics to explore:

- sentence-transformers;
- PyTorch transformer encoders;
- Temporal Fusion Transformer;
- residual learning;
- attention mechanisms;
- time-series cross-validation;
- textual data collection and preprocessing.

Jaron

Jaron will focus on a self-contained textual-data task that can be done remotely during July and August. This is appropriate because he will not be in Bloomington during the period when the main team will work together intensively in person.

Initial tasks include:

- identify timestamped public text sources relevant to MISO and grid stress;
- collect sample MISO operational notices, weather alerts, energy-market headlines, and other public text sources;
- create a simple CSV with timestamp, source, raw text, and basic notes on relevance;
- document where each text item came from and how it may relate to forecast errors;
- begin learning independently about large language models, transformers, text embeddings, and their use in data processing.

Potential sources:

- MISO operational notifications;
- National Weather Service alerts;
- EIA reports;
- Utility Dive;
- local and regional energy news;
- broader public event/news databases, if useful.

Jaron's first goal is not to build the model, but to help create a clean and well-documented textual dataset that the main team can later use in the transformer-based residual-learning pipeline.

13. Learning Resources

Suggested resources:

1. Andrej Karpathy, “Let’s build GPT from scratch.”
2. Jay Alammar, “The Illustrated Transformer.”
3. Introductory tutorials on sentence-transformers.
4. PyTorch tutorials on transformer encoders.
5. Tutorials on Temporal Fusion Transformers.

The goal is not to become deep-learning specialists immediately. The goal is to understand enough to build a controlled, interpretable Stage 3 extension of our existing EML forecaster.

14. First Research Question

The first research question should be:

Does timestamped textual information help explain and reduce the forecast errors of the existing EML Stage 2 load forecaster?

If yes, the next question is:

Does the text help because it contains information already captured by weather and calendar variables, or because it provides genuinely new information?

These are exactly the kinds of questions that connect econometric discipline with modern AI tools.

15. Final Guiding Principle

The transformer should not become an uncontrolled black box. It should be built around the EML principle:

The Stage 2 model provides the interpretable economic structure. The Stage 3 transformer adds flexibility by absorbing richer information streams, but its contribution must be measurable, disciplined, and externally interpretable.

That is the architecture we should build.